

```

const ComponentFunction = function() {
  // @section:imports @depends:[]
  var React = require('react');
  var useState = React.useState;
  var useEffect = React.useEffect;
  var useContext = React.useContext;
  var useMemo = React.useMemo;
  var useCallback = React.useCallback;
  var useRef = React.useRef;
  var View = require('react-native').View;
  var Text = require('react-native').Text;
  var StyleSheet = require('react-native').StyleSheet;
  var ScrollView = require('react-native').ScrollView;
  var TouchableOpacity = require('react-native').TouchableOpacity;
  var TextInput = require('react-native').TextInput;
  var Modal = require('react-native').Modal;
  var Alert = require('react-native').Alert;
  var Platform = require('react-native').Platform;
  var StatusBar = require('react-native').StatusBar;
  var FlatList = require('react-native').FlatList;
  var Dimensions = require('react-native').Dimensions;
  var ActivityIndicator = require('react-native').ActivityIndicator;
  var Ionicons = require('@react-native-vector-icons/ionicons').Ionicons;
  var MaterialIcons = require('@react-native-vector-icons/material-icons').MaterialIcons;
  var createBottomTabNavigator =
require('@react-navigation/bottom-tabs').createBottomTabNavigator;
  var createStackNavigator = require('@react-navigation/stack').createStackNavigator;
  var useSafeAreaInsets = require('react-native-safe-area-context').useSafeAreaInsets;
  var useQuery = require('platform-hooks').useQuery;
  var useMutation = require('platform-hooks').useMutation;
  var useStorage = require('platform-hooks').useStorage;
  var useVideo = require('platform-hooks').useVideo;
  // @end:imports

  // @section:theme @depends:[]
  var TAB_MENU_HEIGHT = Platform.OS === 'web' ? 56 : 49;
  var SCROLL_EXTRA_PADDING = 16;
  var WEB_TAB_MENU_PADDING = 90;
  var FAB_SPACING = 16;
  var HEADER_HEIGHT = 60;
  var primaryColor = '#7C3AED';
  var accentColor = '#F59E0B';
  var backgroundColor = '#0F0A1E';
  var cardColor = '#1E1535';
  var textPrimary = '#F9F9FB';
  var textSecondary = '#A78BFA';
  var designStyle = 'glassmorphic';
  var successColor = '#10B981';

```

```

var errorColor = '#EF4444';
var cardBorder = '#2D2060';
// @end:theme

// @section:navigation-setup @depends:[]
var Tab = createBottomTabNavigator();
var Stack = createStackNavigator();
// @end:navigation-setup

// @section:static-data @depends:[]
var PREFERENCE_CATEGORIES = [
  { id: 'anime', label: 'Anime', emoji: '🌀', color: '#E74C3C', description: 'Naruto, One Piece, Demon Slayer...' },
  { id: 'sports', label: 'Sports', emoji: '⚽', color: '#27AE60', description: 'Soccer, Basketball, Tennis...' },
  { id: 'videogames', label: 'Video Games', emoji: '🎮', color: '#2980B9', description: 'Pokemon, Minecraft, Zelda...' },
  { id: 'music', label: 'Music', emoji: '🎵', color: '#8E44AD', description: 'Pop, Hip-Hop, K-Pop...' },
  { id: 'movies', label: 'Movies', emoji: '🎬', color: '#E67E22', description: 'Marvel, Star Wars, Disney...' },
  { id: 'science', label: 'Science', emoji: '🔬', color: '#16A085', description: 'Space, Biology, Chemistry...' },
  { id: 'art', label: 'Art & Craft', emoji: '🎨', color: '#D35400', description: 'Drawing, Photography, Design...' },
  { id: 'nature', label: 'Nature', emoji: '🌿', color: '#1ABC9C', description: 'Animals, Oceans, Geography...' },
];

var THEME_EMOJI_PAIRS = {
  anime: ['🌀', '⚡', '🗡️', '🔥', '🌸', '💥', '👁️', '🦊', '🌙', '🏯', '🐉', '🌊'],
  sports: ['⚽', '🏀', '🎾', '🏈', '🏊', '🏏', '🏐', '🏑', '🚴', '🏆', '🏅', '💡'],
  videogames: ['🎮', '🕹️', '🏆', '🎯', '🛡️', '👾', '🎮', '🎯', '💣', '☀️'],
  music: ['🎵', '🎸', '🎹', '🎺', '🥁', '🎸', '🎤', '🎧', '🎵', '🎸', '🎺', '🥁'],
  movies: ['🎬', '🦸', '🚀', '🧙', '🦁', '👾', '🦄', '🦉', '🦇', '🪄', '🦑'],
  science: ['🔬', '🚀', '⚙️', '🧬', '🚀', '🦑', '🔬', '🌍', '💡', '🧲', '🌡️', '🧪'],
  art: ['🎨', '🖌️', '🖋️', '🖼️', '📷', '🏺', '✂️', '🎨', '📐', '🧸', '🍪'],
  nature: ['🌿', '🦁', '🦋', '🦋', '🌸', '🌊', '🦅', '🐘', '🌋', '🌈', '🍄', '🦜'],
};

var SOCIAL_CHALLENGES = [
  { id: 'ch1', text: 'Say hello to someone new in class today', category: 'conversation', emoji: '👋', points: 10 },
  { id: 'ch2', text: 'Give a genuine compliment to a classmate', category: 'positivity', emoji: '💬', points: 10 },
  { id: 'ch3', text: 'Ask someone about their favorite hobby', category: 'curiosity', emoji: '🤔', points: 15 },
];

```

```

    { id: 'ch4', text: 'Share something you learned today with a friend', category: 'sharing',
      emoji: '📖', points: 15 },
    { id: 'ch5', text: 'Listen actively when someone is talking to you', category: 'listening', emoji:
      '👂', points: 10 },
    { id: 'ch6', text: 'Offer to help a classmate with something', category: 'helping', emoji: '🤝',
      points: 20 },
    { id: 'ch7', text: 'Join a group activity or study session', category: 'teamwork', emoji: '👥',
      points: 20 },
    { id: 'ch8', text: 'Express your opinion respectfully in a discussion', category: 'confidence',
      emoji: '💪', points: 15 },
    { id: 'ch9', text: 'Introduce yourself to someone you have not spoken to', category:
      'confidence', emoji: '🌟', points: 20 },
    { id: 'ch10', text: 'Thank your teacher or a friend for something specific', category:
      'gratitude', emoji: '🙏', points: 10 },
    { id: 'ch11', text: 'Start a conversation about something you both like', category:
      'conversation', emoji: '💡', points: 15 },
    { id: 'ch12', text: 'Encourage someone who seems stressed or upset', category: 'empathy',
      emoji: '❤️', points: 20 },
    { id: 'ch13', text: 'Disagree politely with something you heard today', category: 'confidence',
      emoji: '🗣️', points: 20 },
    { id: 'ch14', text: 'Ask a classmate to study with you', category: 'teamwork', emoji: '📚',
      points: 15 },
    { id: 'ch15', text: 'Share a fun fact about your favorite interest', category: 'sharing', emoji:
      '🎉', points: 10 },
  ];

```

```

var CATEGORY_BADGES = [
  { id: 'first_flash', label: 'Flashcard Fan', emoji: '📖', condition: 'Create your first flashcard
  set' },
  { id: 'memory_master', label: 'Memory Master', emoji: '🧠', condition: 'Complete a memory
  game' },
  { id: 'social_star', label: 'Social Star', emoji: '🌟', condition: 'Complete 5 social challenges'
  },
  { id: 'streak_7', label: '7-Day Streak', emoji: '🔥', condition: 'Study 7 days in a row' },
  { id: 'scholar', label: 'Scholar', emoji: '🎓', condition: 'Study 10 different topics' },
];
// @end:static-data

```

```
// @section:ThemeContext @depends:[theme]
```

```

var ThemeContext = React.createContext({
  theme: {
    colors: {
      primary: primaryColor, accent: accentColor, background: backgroundColor,
      card: cardColor, textPrimary: textPrimary, textSecondary: textSecondary,
      border: cardBorder, success: successColor, error: errorColor
    }
  },
  preferences: [],

```

```

    darkMode: true
  });

var ThemeProvider = function(props) {
  var theme = useMemo(function() {
    return {
      colors: {
        primary: primaryColor, accent: accentColor, background: backgroundColor,
        card: cardColor, textPrimary: textPrimary, textSecondary: textSecondary,
        border: cardBorder, success: successColor, error: errorColor
      }
    };
  }, []);
  var value = useMemo(function() {
    return { theme: theme, darkMode: true };
  }, [theme]);
  return React.createElement(ThemeContext.Provider, { value: value }, props.children);
};

var useTheme = function() { return useContext(ThemeContext); };
// @end:ThemeContext

// @section:RootScreen @depends:[ThemeContext]
var RootScreen = function(props) {
  var navigation = props.navigation;
  var storedResult = useStorage('onboarding_complete', false);
  var onboardingComplete = storedResult[0];
  var onboardingMeta = storedResult[2];

  useEffect(function() {
    if (!onboardingMeta.ready) return;
    if (onboardingComplete) {
      navigation.replace('Main');
    } else {
      navigation.replace('Onboarding');
    }
  }, [onboardingComplete, onboardingMeta.ready]);

  return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor,
    alignItems: 'center', justifyContent: 'center' }, componentId: 'root-loading' },
    React.createElement(Text, { style: { fontSize: 48, marginBottom: 16 } }, '🌊'),
    React.createElement(Text, { style: { color: primaryColor, fontSize: 28, fontWeight: '900',
    letterSpacing: 2 } }, 'LearnFlow'),
    React.createElement(ActivityIndicator, { color: primaryColor, size: 'large', style: {
    marginTop: 24 } })
  );
};
// @end:RootScreen

```

```

// @section:OnboardingScreen-state @depends:[ThemeContext]
var useOnboardingState = function() {
  var themeCtx = useTheme();
  var theme = themeCtx.theme;
  var selectedState = useState([]);
  var selected = selectedState[0];
  var setSelected = selectedState[1];
  var stepState = useState(0);
  var step = stepState[0];
  var setStep = stepState[1];
  return { theme: theme, selected: selected, setSelected: setSelected, step: step, setStep:
setStep };
};
// @end:OnboardingScreen-state

// @section:OnboardingScreen @depends:[OnboardingScreen-state,static-data]
var OnboardingScreen = function(props) {
  var navigation = props.navigation;
  var state = useOnboardingState();
  var insets = useSafeAreaInsets();
  var prefsStorage = useStorage('user_preferences', { categories: [], selectedThemes: [] });
  var setPrefs = prefsStorage[1];
  var completeStorage = useStorage('onboarding_complete', false);
  var setComplete = completeStorage[1];

  var togglePref = function(id) {
    state.setSelected(function(prev) {
      if (prev.indexOf(id) >= 0) {
        return prev.filter(function(x) { return x !== id; });
      }
      return prev.concat([id]);
    });
  };

  var handleFinish = function() {
    if (state.selected.length < 2) {
      if (Platform.OS === 'web') { window.alert('Please select at least 2 interests!'); }
      else { Alert.alert('Almost there!', 'Please select at least 2 interests to personalize your
experience.');
```

```

    setComplete(true).then(function() {
      navigation.replace('Main');
    });
  };

  var screenH = Dimensions.get('window').height;

  return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor },
    componentId: 'onboarding-screen' },
    React.createElement(StatusBar, { barStyle: 'light-content', backgroundColor:
    backgroundColor }),
    React.createElement(ScrollView, {
      contentContainerStyle: { paddingTop: insets.top + 24, paddingBottom: insets.bottom +
    40, paddingHorizontal: 20 },
      showsVerticalScrollIndicator: false
    },
      React.createElement(View, { style: { alignItems: 'center', marginBottom: 32 },
        componentId: 'onboarding-header' },
        React.createElement(Text, { style: { fontSize: 56 } }, '🌊'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 32, fontWeight: '900',
        marginTop: 8, letterSpacing: 1 } }, 'LearnFlow'),
        React.createElement(View, { style: { width: 80, height: 3, backgroundColor:
        primaryColor, borderRadius: 2, marginTop: 8 } },
          React.createElement(Text, { style: { color: textSecondary, fontSize: 16, textAlign:
        'center', marginTop: 16, lineHeight: 24 } }, 'Pick your interests so we can\nmake learning epic
        for YOU! 🚀')
        ),
          React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '700',
          marginBottom: 6 }, componentId: 'onboarding-subtitle' },
            'What are you into? ' + (state.selected.length > 0 ? '(' + state.selected.length + '
            selected)' : '')
          ),
          React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom:
          20 } }, 'Select at least 2 interests'),
          React.createElement(View, { style: { flexDirection: 'row', flexWrap: 'wrap', justifyContent:
          'space-between' }, componentId: 'prefs-grid' },
            PREFERENCE_CATEGORIES.map(function(cat) {
              var isSelected = state.selected.indexOf(cat.id) >= 0;
              return React.createElement(TouchableOpacity, {
                key: cat.id,
                onPress: function() { togglePref(cat.id); },
                style: {
                  width: '48%', marginBottom: 12,
                  borderRadius: 16, padding: 16,
                  backgroundColor: isSelected ? cat.color + '33' : cardColor,
                  borderWidth: 2, borderColor: isSelected ? cat.color : cardBorder,
                  alignItems: 'center'
                }
              ),
            }
          ),

```

```

        componentId: 'pref-' + cat.id
      },
      React.createElement(View, { style: { width: 52, height: 52, borderRadius: 26,
backgroundColor: cat.color + '22', alignItems: 'center', justifyContent: 'center', marginBottom:
8 } }},
        React.createElement(Text, { style: { fontSize: 28 } }, cat.emoji)
      ),
      React.createElement(Text, { style: { color: isSelected ? cat.color : textPrimary,
fontWeight: '700', fontSize: 15, marginBottom: 4 } }, cat.label),
      React.createElement(Text, { style: { color: textSecondary, fontSize: 11, textAlign:
'center' } }, cat.description),
      isSelected ? React.createElement(View, { style: { position: 'absolute', top: 8, right: 8,
backgroundColor: cat.color, borderRadius: 10, width: 20, height: 20, alignItems: 'center',
justifyContent: 'center' } },
        React.createElement(Text, { style: { color: '#fff', fontSize: 12, fontWeight: '700' } },
'✓')
      ) : null
    );
  })
),
React.createElement(TouchableOpacity, {
  onPress: handleFinish,
  style: {
    marginTop: 16, backgroundColor: state.selected.length >= 2 ? primaryColor :
cardColor,
    borderRadius: 16, padding: 18, alignItems: 'center',
    borderWidth: state.selected.length >= 2 ? 0 : 1, borderColor: cardBorder
  },
  componentId: 'onboarding-continue-btn'
}),
React.createElement(Text, { style: { color: state.selected.length >= 2 ? '#fff' :
textSecondary, fontSize: 18, fontWeight: '800' } },
  state.selected.length >= 2 ? 'Start Learning! 🚀' : 'Select at least 2 to continue'
)
)
);
};
// @end:OnboardingScreen

// @section:HomeScreen-state @depends:[ThemeContext]
var useHomeState = function() {
  var themeCtx = useTheme();
  var theme = themeCtx.theme;
  var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });
  var prefs = prefsResult[0];
  var sessionsQuery = useQuery('study_sessions', {}, { column: 'session_date', ascending:
false });

```

```

    var sessions = sessionsQuery.data;
    var sessionsLoading = sessionsQuery.loading;
    var flashQuery = useQuery('flashcard_sets', {}, { column: 'created_at_local', ascending:
false });
    var flashSets = flashQuery.data;
    var streakState = useState(0);
    var streak = streakState[0];
    var setStreak = streakState[1];
    var totalPointsState = useState(0);
    var totalPoints = totalPointsState[0];
    var setTotalPoints = totalPointsState[1];

    useEffect(function() {
      if (!sessions || sessions.length === 0) { setStreak(0); setTotalPoints(0); return; }
      var points = 0;
      sessions.forEach(function(s) { points += (s.score || 0); });
      setTotalPoints(points);
      var today = new Date().toISOString().slice(0, 10);
      var uniqueDates = [];
      sessions.forEach(function(s) { if (uniqueDates.indexOf(s.session_date) < 0) {
uniqueDates.push(s.session_date); } });
      uniqueDates.sort().reverse();
      var currentStreak = 0;
      var checkDate = new Date();
      for (var i = 0; i < uniqueDates.length; i++) {
        var d = checkDate.toISOString().slice(0, 10);
        if (uniqueDates[i] === d) {
          currentStreak++;
          checkDate.setDate(checkDate.getDate() - 1);
        } else { break; }
      }
      setStreak(currentStreak);
    }, [sessions]);

    return { theme: theme, prefs: prefs, sessions: sessions, sessionsLoading:
sessionsLoading, flashSets: flashSets, streak: streak, totalPoints: totalPoints };
  };
  // @end:HomeScreen-state

  // @section:HomeScreen @depends:[HomeScreen-state,static-data]
  var HomeScreen = function(props) {
    var navigation = props.navigation;
    var state = useHomeState();
    var insets = useSafeAreaInsets();
    var scrollTopPadding = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + SCROLL_EXTRA_PADDING);
    var videoHook = useVideo();
    var play = videoHook.play;

```



```

var pause = videoHook.pause;
var isPlaying = videoHook.isPlaying;
var VideoPlayer = videoHook.VideoPlayer;
var showVideoState = useState(false);
var showVideo = showVideoState[0];
var setShowVideo = showVideoState[1];

var userCats = state.prefs && state.prefs.categories ? state.prefs.categories : [];
var greeting = function() {
  var h = new Date().getHours();
  if (h < 12) return 'Good Morning! 🌞';
  if (h < 17) return 'Good Afternoon! 🌤️';
  return 'Good Evening! 🌙';
};

var recentSessions = (state.sessions && state.sessions.length > 0) ?
state.sessions.slice(0, 3) : [];
var recentSets = (state.flashSets && state.flashSets.length > 0) ? state.flashSets.slice(0,
3) : [];

var handleVideoPress = function() {
  if (!showVideo) {
    setShowVideo(true);
    play('https://thinkablecdn.com/example-video/Big_Buck_Bunny_1080_10s_1MB.mp4');
  } else {
    if (isPlaying) { pause(); } else { play(); }
  }
};

return React.createElement(ScrollView, {
  style: { flex: 1, backgroundColor: backgroundColor },
  contentContainerStyle: { paddingTop: insets.top + 16, paddingBottom:
scrollBottomPadding, paddingHorizontal: 16 },
  showsVerticalScrollIndicator: false,
  componentId: 'home-screen'
},
  React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between', marginBottom: 20 }, componentId: 'home-header' },
    React.createElement(View, {},
      React.createElement(Text, { style: { color: textSecondary, fontSize: 14 } }, greeting()),
      React.createElement(Text, { style: { color: textPrimary, fontSize: 24, fontWeight: '900' }
}, 'LearnFlow 🐙')
    ),
    React.createElement(View, { style: { alignItems: 'center', backgroundColor: cardColor,
borderRadius: 14, padding: 10, borderWidth: 1, borderColor: cardBorder } },
      React.createElement(Text, { style: { fontSize: 22 } }, '🔥'),
      React.createElement(Text, { style: { color: accentColor, fontSize: 18, fontWeight: '900' }
}, String(state.streak)),

```

```

    React.createElement(Text, { style: { color: textSecondary, fontSize: 10 } }, 'streak')
  )
),

    React.createElement(View, { style: { flexDirection: 'row', marginBottom: 20, gap: 12 },
    componentId: 'stats-row' },
      React.createElement(View, { style: { flex: 1, backgroundColor: primaryColor + '22',
borderRadius: 16, padding: 16, borderWidth: 1, borderColor: primaryColor + '44', alignItems:
'center' } },
        React.createElement(Text, { style: { fontSize: 28 } }, '⚡'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 22, fontWeight: '900' }
}, String(state.totalPoints)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Total XP')
      ),
      React.createElement(View, { style: { flex: 1, backgroundColor: accentColor + '22',
borderRadius: 16, padding: 16, borderWidth: 1, borderColor: accentColor + '44', alignItems:
'center' } },
        React.createElement(Text, { style: { fontSize: 28 } }, '🎮'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 22, fontWeight: '900' }
}, String(state.flashSets ? state.flashSets.length : 0)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Card Sets')
      ),
      React.createElement(View, { style: { flex: 1, backgroundColor: successColor + '22',
borderRadius: 16, padding: 16, borderWidth: 1, borderColor: successColor + '44', alignItems:
'center' } },
        React.createElement(Text, { style: { fontSize: 28 } }, '🎮'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 22, fontWeight: '900' }
}, String(state.sessions ? state.sessions.length : 0)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Sessions')
      )
    ),

    React.createElement(View, { style: { marginBottom: 20 }, componentId: 'quick-actions' },
      React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '⚡ Quick Actions'),
      React.createElement(View, { style: { flexDirection: 'row', gap: 12 } },
        React.createElement(TouchableOpacity, {
          onPress: function() { navigation.navigate('CreateSet'); },
          style: { flex: 1, backgroundColor: primaryColor, borderRadius: 16, padding: 16,
alignItems: 'center' },
          componentId: 'home-create-btn'
        },
          React.createElement(Text, { style: { fontSize: 24, marginBottom: 4 } }, '⚡'),
          React.createElement(Text, { style: { color: '#fff', fontWeight: '700', fontSize: 13,
textAlign: 'center' } }, 'New Flash Set')
        ),
        React.createElement(TouchableOpacity, {
          onPress: function() { navigation.navigate('Games'); },

```

```

        style: { flex: 1, backgroundColor: '#2980B9', borderRadius: 16, padding: 16,
alignItems: 'center' },
        componentId: 'home-games-btn'
    },
    React.createElement(Text, { style: { fontSize: 24, marginBottom: 4 } }, '🎮'),
    React.createElement(Text, { style: { color: '#fff', fontWeight: '700', fontSize: 13,
textAlign: 'center' } }, 'Play Games')
),
    React.createElement(TouchableOpacity, {
        onPress: function() { navigation.navigate('Challenges'); },
        style: { flex: 1, backgroundColor: '#16A085', borderRadius: 16, padding: 16,
alignItems: 'center' },
        componentId: 'home-challenges-btn'
    },
    React.createElement(Text, { style: { fontSize: 24, marginBottom: 4 } }, '🏆'),
    React.createElement(Text, { style: { color: '#fff', fontWeight: '700', fontSize: 13,
textAlign: 'center' } }, 'Daily Quest')
    )
    )
),

    React.createElement(View, { style: { marginBottom: 20 }, componentId: 'study-tip-video' },
        React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '📺 Daily Study Tip'),
        React.createElement(TouchableOpacity, {
            onPress: handleVideoPress,
            style: { backgroundColor: cardColor, borderRadius: 16, overflow: 'hidden',
borderWidth: 1, borderColor: cardBorder },
            componentId: 'video-tip-btn'
        },
            showVideo
            ? React.createElement(View, { style: { height: 200 } },
                React.createElement(VideoPlayer, { style: { width: '100%', height: 200 } }),
                React.createElement(View, { style: { position: 'absolute', bottom: 8, right: 8,
backgroundColor: 'rgba(0,0,0,0.6)', borderRadius: 20, paddingHorizontal: 12,
paddingVertical: 6 } },
                    React.createElement(Text, { style: { color: '#fff', fontSize: 12 } }, isPlaying ? '⏸
Pause' : '▶ Play')
                )
            )
            : React.createElement(View, { style: { height: 140, alignItems: 'center', justifyContent:
'center', gap: 8 } },
                React.createElement(View, { style: { width: 60, height: 60, borderRadius: 30,
backgroundColor: primaryColor + '33', alignItems: 'center', justifyContent: 'center',
borderWidth: 2, borderColor: primaryColor } },
                    React.createElement(Text, { style: { fontSize: 26 } }, '▶')
                ),

```

```

    React.createElement(Text, { style: { color: textSecondary, fontSize: 14 } }, 'Tap to
watch today's study tip')
  )
)
),

recentSets.length > 0
? React.createElement(View, { style: { marginBottom: 20 }, componentId: 'recent-sets' },
  React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, ' Recent Sets'),
  recentSets.map(function(set) {
    var cat = PREFERENCE_CATEGORIES.find(function(c) { return c.id ===
set.theme; });
    return React.createElement(TouchableOpacity, {
      key: set.id || set.title,
      onPress: function() { navigation.navigate('StudySet', { setId: set.id }); },
      style: { backgroundColor: cardColor, borderRadius: 14, padding: 14,
marginBottom: 10, borderWidth: 1, borderColor: cat ? cat.color + '44' : cardBorder,
flexDirection: 'row', alignItems: 'center' },
      componentId: 'recent-set-' + set.id
    },
      React.createElement(View, { style: { width: 44, height: 44, borderRadius: 22,
backgroundColor: cat ? cat.color + '33' : primaryColor + '33', alignItems: 'center',
justifyContent: 'center', marginRight: 12 } },
        React.createElement(Text, { style: { fontSize: 24 } }, cat ? cat.emoji : '')
      ),
      React.createElement(View, { style: { flex: 1 } },
        React.createElement(Text, { style: { color: textPrimary, fontSize: 15, fontWeight:
'700' } }, set.title),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } },
set.topic + ' • ' + (set.cards_data ? JSON.parse(typeof set.cards_data === 'string' ?
set.cards_data : JSON.stringify(set.cards_data)).length : 0) + ' cards')
      ),
      React.createElement(Text, { style: { color: primaryColor, fontSize: 20 } }, '▶')
    );
  })
)
: React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 16,
padding: 24, alignItems: 'center', marginBottom: 20, borderWidth: 1, borderColor: cardBorder
}, componentId: 'empty-home' },
  React.createElement(Text, { style: { fontSize: 48, marginBottom: 8 } }, ''),
  React.createElement(Text, { style: { color: textPrimary, fontSize: 16, fontWeight: '700',
marginBottom: 4 } }, 'Ready to grow?'),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 13, textAlign:
'center' } }, 'Create your first flashcard set or play a game to get started!')
),

React.createElement(View, { style: { marginBottom: 8 }, componentId: 'badges-row' },

```

```

    React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '🏆 Badges'),
    React.createElement(ScrollView, { horizontal: true, showsHorizontalScrollIndicator:
false, style: { flexGrow: 'initial' } },
    CATEGORY_BADGES.map(function(badge) {
    return React.createElement(View, {
    key: badge.id,
    style: { backgroundColor: cardColor, borderRadius: 14, padding: 14, marginRight:
10, alignItems: 'center', width: 90, borderWidth: 1, borderColor: cardBorder },
    componentId: 'badge-' + badge.id
    },
    React.createElement(Text, { style: { fontSize: 32, marginBottom: 4 } }, badge.emoji),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 10, textAlign:
'center' } }, badge.label)
    );
    })
    )
    );
};
// @end:HomeScreen

```

```

// @section:FlashcardsScreen-state @depends:[ThemeContext]
var useFlashcardsState = function() {
    var themeCtx = useTheme();
    var theme = themeCtx.theme;
    var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });
    var prefs = prefsResult[0];
    var flashQuery = useQuery('flashcard_sets', {}, { column: 'created_at_local', ascending:
false });
    var flashSets = flashQuery.data;
    var flashLoading = flashQuery.loading;
    var flashRefetch = flashQuery.refetch;
    var deleteMutation = useMutation('flashcard_sets', 'delete');
    var deleteMutate = deleteMutation.mutate;
    var searchState = useState("");
    var search = searchState[0];
    var setSearch = searchState[1];
    return { theme: theme, prefs: prefs, flashSets: flashSets, flashLoading: flashLoading,
flashRefetch: flashRefetch, deleteMutate: deleteMutate, search: search, setSearch:
setSearch };
};
// @end:FlashcardsScreen-state

```

```

// @section:FlashcardsScreen @depends:[FlashcardsScreen-state]
var FlashcardsScreen = function(props) {
    var navigation = props.navigation;
    var state = useFlashcardsState();

```

```

var insets = useSafeAreaInsets();
var scrollBottomPadding = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + SCROLL_EXTRA_PADDING);
var fabBottom = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + FAB_SPACING);

var filteredSets = useMemo(function() {
  var sets = (state.flashSets && state.flashSets.length > 0) ? state.flashSets : [];
  if (!state.search) return sets;
  var q = state.search.toLowerCase();
  return sets.filter(function(s) { return s.title.toLowerCase().indexOf(q) >= 0 ||
s.topic.toLowerCase().indexOf(q) >= 0; });
}, [state.flashSets, state.search]);

var handleDelete = function(id) {
  if (Platform.OS === 'web') {
    if (!window.confirm('Delete this flashcard set?')) return;
    state.deleteMutate({ id: id }).then(function() { state.flashRefetch(); });
  } else {
    Alert.alert('Delete Set', 'Are you sure?', [
      { text: 'Cancel', style: 'cancel' },
      { text: 'Delete', style: 'destructive', onPress: function() { state.deleteMutate({ id: id
}).then(function() { state.flashRefetch(); }); } }
    ]);
  }
};

var getCardCount = function(cardsData) {
  try {
    var arr = typeof cardsData === 'string' ? JSON.parse(cardsData) : cardsData;
    return Array.isArray(arr) ? arr.length : 0;
  } catch(e) { return 0; }
};

return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor },
componentId: 'flashcards-screen' },
  React.createElement(View, { style: { paddingTop: insets.top + 12, paddingBottom: 12,
paddingHorizontal: 16, backgroundColor: backgroundColor, borderBottomWidth: 1,
borderBottomColor: cardBorder } },
    React.createElement(Text, { style: { color: textPrimary, fontSize: 22, fontWeight: '900',
marginBottom: 10 } }, '📚 My Flashcard Sets'),
    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
backgroundColor: cardColor, borderRadius: 12, paddingHorizontal: 12, borderWidth: 1,
borderColor: cardBorder } },
      React.createElement(Text, { style: { color: textSecondary, marginRight: 8 } }, '🔍'),
      React.createElement(TextInput, {
        value: state.search,
        onChangeText: state.setSearch,

```

```

placeholder: 'Search sets...',
placeholderTextColor: textSecondary,
style: { flex: 1, color: textPrimary, height: 40, fontSize: 15 },
autoCapitalize: 'none',
componentId: 'flash-search-input'
  })
)
),
state.flashLoading
  ? React.createElement(ActivityIndicator, { color: primaryColor, size: 'large', style: { flex:
1 }})
: React.createElement(ScrollView, {
  style: { flex: 1 },
  contentContainerStyle: { paddingTop: 16, paddingBottom: scrollBottomPadding,
paddingHorizontal: 16 },
  showsVerticalScrollIndicator: false
},
  filteredSets.length === 0
    ? React.createElement(View, { style: { alignItems: 'center', paddingTop: 60 },
componentId: 'flash-empty' },
      React.createElement(Text, { style: { fontSize: 64, marginBottom: 16 } }, '🎉'),
      React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight:
'700', marginBottom: 8 } }, 'No flashcard sets yet!'),
      React.createElement(Text, { style: { color: textSecondary, fontSize: 14, textAlign:
'center' } }, 'Tap the + button below to create your first set')
    )
    : filteredSets.map(function(set) {
      var cat = PREFERENCE_CATEGORIES.find(function(c) { return c.id ===
set.theme; });
      var cardCount = getCardCount(set.cards_data);
      return React.createElement(TouchableOpacity, {
        key: set.id,
        onPress: function() { navigation.navigate('StudySet', { setId: set.id }); },
        style: { backgroundColor: cardColor, borderRadius: 18, padding: 18,
marginBottom: 14, borderWidth: 1, borderColor: cat ? cat.color + '55' : cardBorder },
        componentId: 'flash-set-' + set.id
      },
        React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
marginBottom: 10 } },
          React.createElement(View, { style: { width: 52, height: 52, borderRadius: 14,
backgroundColor: cat ? cat.color + '33' : primaryColor + '33', alignItems: 'center',
justifyContent: 'center', marginRight: 12 } },
            React.createElement(Text, { style: { fontSize: 28 } }, cat ? cat.emoji : '🎉')
          ),
          React.createElement(View, { style: { flex: 1 } },
            React.createElement(Text, { style: { color: textPrimary, fontSize: 17,
fontWeight: '800' } }, set.title),

```

```

        React.createElement(Text, { style: { color: textSecondary, fontSize: 13 } },
set.topic)
    ),
    React.createElement(TouchableOpacity, {
        onPress: function() { handleDelete(set.id); },
        style: { padding: 8 },
        componentId: 'flash-delete-' + set.id
    },
        React.createElement(Text, { style: { color: errorColor, fontSize: 16 } }, '🗑️')
    ),
    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between' } },
        React.createElement(View, { style: { flexDirection: 'row', gap: 8 } },
            React.createElement(View, { style: { backgroundColor: primaryColor + '33',
borderRadius: 8, paddingHorizontal: 10, paddingVertical: 4 } },
                React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } },
cardCount + ' cards')
            ),
            cat ? React.createElement(View, { style: { backgroundColor: cat.color + '22',
borderRadius: 8, paddingHorizontal: 10, paddingVertical: 4 } },
                React.createElement(Text, { style: { color: cat.color, fontSize: 12, fontWeight:
'600' } }, cat.label)
            ) : null
        ),
        React.createElement(Text, { style: { color: primaryColor, fontSize: 14,
fontWeight: '700' } }, 'Study →')
    ),
    ),
    React.createElement(TouchableOpacity, {
        onPress: function() { navigation.navigate('CreateSet'); },
        style: { position: 'absolute', bottom: fabBottom, right: 20, width: 58, height: 58,
borderRadius: 29, backgroundColor: primaryColor, alignItems: 'center', justifyContent:
'center', shadowColor: primaryColor, shadowOffset: { width: 0, height: 4 }, shadowOpacity:
0.5, shadowRadius: 8, elevation: 8 },
        componentId: 'flash-fab-btn'
    },
        React.createElement(Text, { style: { color: '#fff', fontSize: 30, fontWeight: '300',
marginTop: -2 } }, '+')
    ),
    );
};
// @end:FlashcardsScreen

// @section:CreateSetScreen-state @depends:[ThemeContext]
var useCreateSetState = function() {

```



```

var themeCtx = useTheme();
var theme = themeCtx.theme;
var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });
var prefs = prefsResult[0];
var titleState = useState("");
var title = titleState[0];
var setTitle = titleState[1];
var topicState = useState("");
var topic = topicState[0];
var setTopic = topicState[1];
var themeState = useState("");
var selectedTheme = themeState[0];
var setSelectedTheme = themeState[1];
var cardsState = useState([ { id: 'c1', question: " ", answer: " " } ]);
var cards = cardsState[0];
var setCards = cardsState[1];
return { theme: theme, prefs: prefs, title: title, setTitle: setTitle, topic: topic, setTopic:
setTopic, selectedTheme: selectedTheme, setSelectedTheme: setSelectedTheme, cards:
cards, setCards: setCards };
};
// @end:CreateSetScreen-state

```

```

// @section:CreateSetScreen @depends:[CreateSetScreen-state]

```

```

var CreateSetScreen = function(props) {
  var navigation = props.navigation;
  var state = useCreateSetState();
  var insets = useSafeAreaInsets();
  var insertMutation = useMutation('flashcard_sets', 'insert');
  var insertMutate = insertMutation.mutate;
  var sessionMutation = useMutation('study_sessions', 'insert');
  var sessionMutate = sessionMutation.mutate;
  var screenH = Dimensions.get('window').height;
  var scrollH = screenH - HEADER_HEIGHT - insets.top;

  var addCard = function() {
    state.setCards(function(prev) {
      return prev.concat([ { id: 'c' + Date.now(), question: " ", answer: " " } ]);
    });
  };

  var updateCard = function(id, field, value) {
    state.setCards(function(prev) {
      return prev.map(function(c) {
        if (c.id !== id) return c;
        var updated = Object.assign({}, c);
        updated[field] = value;
        return updated;
      });
    });
  };

```

```

    });
};

var removeCard = function(id) {
    state.setCards(function(prev) { return prev.filter(function(c) { return c.id !== id; }); });
};

var handleSave = function() {
    if (!state.title.trim()) {
        if (Platform.OS === 'web') { window.alert('Please add a title!'); } else {
Alert.alert('Missing title', 'Please add a title for your set.');
```

```

    return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor },
    componentId: 'create-set-screen' },
    React.createElement(View, { style: { height: HEADER_HEIGHT + insets.top, paddingTop:
    insets.top, backgroundColor: cardColor, flexDirection: 'row', alignItems: 'center',
    paddingHorizontal: 16, borderBottomWidth: 1, borderBottomColor: cardBorder } },
    React.createElement(TouchableOpacity, { onPress: function() { navigation.goBack(); },
    style: { marginRight: 12, padding: 4 }, componentId: 'create-back-btn' },
    React.createElement(Text, { style: { color: primaryColor, fontSize: 24 } }, '←')
    ),
    React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
    flex: 1 } }, '🌟 Create Flashcard Set'),
    React.createElement(TouchableOpacity, { onPress: handleSave, style: {
    backgroundColor: primaryColor, borderRadius: 10, paddingHorizontal: 16, paddingVertical: 8
    }, componentId: 'create-save-btn' },
    React.createElement(Text, { style: { color: '#fff', fontWeight: '700' } }, 'Save')
    )
    ),
    React.createElement(ScrollView, {
    style: Platform.OS === 'web' ? { height: scrollH, overflow: 'auto' } : { flex: 1 },
    contentContainerStyle: { paddingTop: 16, paddingBottom: insets.bottom + 40,
    paddingHorizontal: 16 },
    showsVerticalScrollIndicator: false
    },
    React.createElement(View, { style: { marginBottom: 16 }, componentId:
    'create-title-section' },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 13, fontWeight:
    '600', marginBottom: 6, textTransform: 'uppercase', letterSpacing: 1 } }, 'Set Title *'),
    React.createElement(TextInput, {
    value: state.title,
    onChangeText: state.setTitle,
    placeholder: 'e.g. Biology Chapter 3...',
    placeholderTextColor: textSecondary,
    style: { backgroundColor: cardColor, borderRadius: 12, padding: 14, color:
    textPrimary, fontSize: 16, borderWidth: 1, borderColor: cardBorder },
    autoCapitalize: 'words',
    componentId: 'create-title-input'
    })
    ),
    React.createElement(View, { style: { marginBottom: 16 }, componentId:
    'create-topic-section' },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 13, fontWeight:
    '600', marginBottom: 6, textTransform: 'uppercase', letterSpacing: 1 } }, 'Subject / Topic'),
    React.createElement(TextInput, {
    value: state.topic,
    onChangeText: state.setTopic,
    placeholder: 'e.g. Cell Biology, Algebra, History...',
    placeholderTextColor: textSecondary,

```

```

        style: { backgroundColor: cardColor, borderRadius: 12, padding: 14, color:
textPrimary, fontSize: 16, borderWidth: 1, borderColor: cardBorder },
        autoCapitalize: 'words',
        componentId: 'create-topic-input'
    })
),
    React.createElement(View, { style: { marginBottom: 20 }, componentId:
'create-theme-section' },
        React.createElement(Text, { style: { color: textSecondary, fontSize: 13, fontWeight:
'600', marginBottom: 8, textTransform: 'uppercase', letterSpacing: 1 } }, '🎨 Theme Your
Set'),
        React.createElement(ScrollView, { horizontal: true, showsHorizontalScrollIndicator:
false, style: { flexGrow: 'initial' } },
            availableThemes.map(function(catId) {
                var cat = PREFERENCE_CATEGORIES.find(function(c) { return c.id === catId; });
                if (!cat) return null;
                var isSelected = state.selectedTheme === catId;
                return React.createElement(TouchableOpacity, {
                    key: catId,
                    onPress: function() { state.setSelectedTheme(catId); },
                    style: { backgroundColor: isSelected ? cat.color : cardColor, borderRadius: 12,
paddingHorizontal: 14, paddingVertical: 10, marginRight: 10, alignItems: 'center',
borderWidth: 1, borderColor: isSelected ? cat.color : cardBorder, flexDirection: 'row', gap: 6 },
                    componentId: 'theme-btn-' + catId
                },
                    React.createElement(Text, { style: { fontSize: 18 } }, cat.emoji),
                    React.createElement(Text, { style: { color: isSelected ? '#fff' : textSecondary,
fontWeight: '600', fontSize: 13 } }, cat.label)
                );
            })
        ),
        React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between', marginBottom: 12 } },
            React.createElement(Text, { style: { color: textPrimary, fontSize: 16, fontWeight: '800' }
}, '📝 Cards (' + state.cards.length + ')'),
            React.createElement(TouchableOpacity, { onPress: addCard, style: {
backgroundColor: primaryColor + '33', borderRadius: 10, paddingHorizontal: 14,
paddingVertical: 8, flexDirection: 'row', alignItems: 'center', gap: 6 }, componentId:
'add-card-btn' },
                React.createElement(Text, { style: { color: primaryColor, fontWeight: '700' } }, '+ Add
Card')
            )
        ),
        state.cards.map(function(card, idx) {
            return React.createElement(View, {
                key: card.id,

```

```

        style: { backgroundColor: cardColor, borderRadius: 16, padding: 16, marginBottom:
12, borderWidth: 1, borderColor: cardBorder },
        componentId: 'card-form-' + card.id
    },
    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between', marginBottom: 10 } },
        React.createElement(Text, { style: { color: primaryColor, fontWeight: '700', fontSize:
14 } }, 'Card ' + (idx + 1)),
        state.cards.length > 1
        ? React.createElement(TouchableOpacity, { onPress: function() {
removeCard(card.id); }, style: { padding: 4 }, componentId: 'remove-card-' + card.id },
            React.createElement(Text, { style: { color: errorColor } }, 'X')
        )
        : null
    ),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12,
marginBottom: 6, fontWeight: '600' } }, 'QUESTION'),
    React.createElement(TextInput, {
        value: card.question,
        onChangeText: function(v) { updateCard(card.id, 'question', v); },
        placeholder: 'Enter question...',
        placeholderTextColor: textSecondary,
        style: { backgroundColor: backgroundColor, borderRadius: 10, padding: 12, color:
textPrimary, fontSize: 15, borderWidth: 1, borderColor: cardBorder, marginBottom: 10 },
        multiline: true, numberOfLines: 2, textAlignVertical: 'top',
        componentId: 'card-q-' + card.id
    }),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12,
marginBottom: 6, fontWeight: '600' } }, 'ANSWER'),
    React.createElement(TextInput, {
        value: card.answer,
        onChangeText: function(v) { updateCard(card.id, 'answer', v); },
        placeholder: 'Enter answer...',
        placeholderTextColor: textSecondary,
        style: { backgroundColor: backgroundColor, borderRadius: 10, padding: 12, color:
textPrimary, fontSize: 15, borderWidth: 1, borderColor: cardBorder },
        multiline: true, numberOfLines: 2, textAlignVertical: 'top',
        componentId: 'card-a-' + card.id
    })
    );
    })
    )
    );
};
// @end:CreateSetScreen

// @section:StudySetScreen-state @depends:[ThemeContext]
var useStudySetState = function(setId) {

```

```

var themeCtx = useTheme();
var theme = themeCtx.theme;
var flashQuery = useQuery('flashcard_sets', {}, {});
var flashSets = flashQuery.data;
var setDataState = useState(null);
var setData = setDataState[0];
var setSetData = setDataState[1];
var cardsState = useState([]);
var cards = cardsState[0];
var setCards = cardsState[1];
var currentIndexState = useState(0);
var currentIndex = currentIndexState[0];
var setCurrentIndex = currentIndexState[1];
var flippedState = useState(false);
var flipped = flippedState[0];
var setFlipped = flippedState[1];
var completedState = useState(false);
var completed = completedState[0];
var setCompleted = completedState[1];
var knownState = useState(0);
var known = knownState[0];
var setKnown = knownState[1];

useEffect(function() {
  if (!flashSets || !setId) return;
  var found = flashSets.find(function(s) { return s.id === setId; });
  if (!found) return;
  setSetData(found);
  try {
    var parsed = typeof found.cards_data === 'string' ? JSON.parse(found.cards_data) :
found.cards_data;
    setCards(Array.isArray(parsed) ? parsed : []);
  } catch(e) { setCards([]); }
}, [flashSets, setId]);

return { theme: theme, setData: setData, cards: cards, currentIndex: currentIndex,
setCurrentIndex: setCurrentIndex, flipped: flipped, setFlipped: setFlipped, completed:
completed, setCompleted: setCompleted, known: known, setKnown: setKnown };
};
// @end:StudySetScreen-state

// @section:StudySetScreen @depends:[StudySetScreen-state]
var StudySetScreen = function(props) {
  var navigation = props.navigation;
  var route = props.route;
  var setId = route && route.params ? route.params.setId : null;
  var state = useStudySetState(setId);
  var insets = useSafeAreaInsets();

```

```

var sessionMutation = useMutation('study_sessions', 'insert');
var sessionMutate = sessionMutation.mutate;
var updateMutation = useMutation('flashcard_sets', 'update');
var updateMutate = updateMutation.mutate;
var startTimeRef = useRef(Date.now());

var cat = state.setData ? PREFERENCE_CATEGORIES.find(function(c) { return c.id ===
state.setData.theme; }) : null;

var handleFlip = function() { state.setFlipped(function(prev) { return !prev; }); };

var handleNext = function(wasKnown) {
  if (wasKnown) { state.setKnown(function(prev) { return prev + 1; }); }
  if (state.currentIndex >= state.cards.length - 1) {
    var duration = Math.round((Date.now() - startTimeRef.current) / 1000);
    var finalKnown = wasKnown ? state.known + 1 : state.known;
    var score = Math.round((finalKnown / state.cards.length) * 100);
    var today = new Date().toISOString().slice(0, 10);
    sessionMutate({ session_type: 'flashcard', content_id: setId || 'unknown',
duration_seconds: duration, score: score, session_date: today });
    if (setId) {
      updateMutate({ id: setId, data: { last_reviewed: new Date().toISOString() } });
    }
    state.setCompleted(true);
  } else {
    state.setCurrentIndex(function(prev) { return prev + 1; });
    state.setFlipped(false);
  }
};

var handleRestart = function() {
  state.setCurrentIndex(0);
  state.setFlipped(false);
  state.setCompleted(false);
  state.setKnown(0);
  startTimeRef.current = Date.now();
};

if (!state.setData) {
  return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor,
alignItems: 'center', justifyContent: 'center' }, componentId: 'study-loading' },
    React.createElement(ActivityIndicator, { color: primaryColor, size: 'large' })
  );
}

var currentCard = state.cards[state.currentIndex] || { question: 'No question', answer: 'No
answer' };
var progress = state.cards.length > 0 ? (state.currentIndex + 1) / state.cards.length : 0;

```

```

    if (state.completed) {
      var pct = state.cards.length > 0 ? Math.round((state.known / state.cards.length) * 100) :
0;
      return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor,
alignItems: 'center', justifyContent: 'center', padding: 24 }, componentId: 'study-complete' },
        React.createElement(Text, { style: { fontSize: 64 }, pct >= 80 ? '🏆' : pct >= 60 ? '🎯' :
'💪'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 28, fontWeight: '900',
marginTop: 16, marginBottom: 8 }, pct >= 80 ? 'Excellent!' : pct >= 60 ? 'Good Job!' : 'Keep
Studying!'),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 16, marginBottom:
24, textAlign: 'center' }, 'You got ' + state.known + ' out of ' + state.cards.length + ' correct ('
+ pct + '%)'),
        React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 24, width: '100%', alignItems: 'center', marginBottom: 24, borderWidth: 1,
borderColor: cardBorder },
        React.createElement(Text, { style: { color: accentColor, fontSize: 48, fontWeight: '900' }
}, pct + '%'),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 14 }, 'Score')
        ),
        React.createElement(TouchableOpacity, { onPress: handleRestart, style: {
backgroundColor: primaryColor, borderRadius: 14, paddingVertical: 16, paddingHorizontal:
40, marginBottom: 12 }, componentId: 'study-restart-btn' },
        React.createElement(Text, { style: { color: '#fff', fontSize: 16, fontWeight: '800' }, '🔄
Study Again')
        ),
        React.createElement(TouchableOpacity, { onPress: function() { navigation.goBack(); },
style: { backgroundColor: cardColor, borderRadius: 14, paddingVertical: 16,
paddingHorizontal: 40, borderWidth: 1, borderColor: cardBorder }, componentId:
'study-back-btn' },
        React.createElement(Text, { style: { color: textSecondary, fontSize: 16, fontWeight:
'700' }, '← Back to Sets')
        )
      );
    }

    return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor },
componentId: 'study-screen' },
      React.createElement(View, { style: { height: HEADER_HEIGHT + insets.top, paddingTop:
insets.top, backgroundColor: cardColor, flexDirection: 'row', alignItems: 'center',
paddingHorizontal: 16, borderBottomWidth: 1, borderBottomColor: cardBorder },
        React.createElement(TouchableOpacity, { onPress: function() { navigation.goBack(); },
style: { padding: 4, marginRight: 12 }, componentId: 'study-back-arrow' },
        React.createElement(Text, { style: { color: primaryColor, fontSize: 24 }, '←')
        ),
        React.createElement(View, { style: { flex: 1 } },

```



```

    React.createElement(Text, { style: { color: textPrimary, fontSize: 16, fontWeight: '800' }
}, state.setData.title),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } },
state.currentIndex + 1 + ' of ' + state.cards.length)
  ),
  cat ? React.createElement(Text, { style: { fontSize: 24 } }, cat.emoji) : null
),
  React.createElement(View, { style: { height: 4, backgroundColor: cardColor } },
    React.createElement(View, { style: { height: 4, backgroundColor: primaryColor, width:
(progress * 100) + '%', borderRadius: 2 } })
  ),
  React.createElement(View, { style: { flex: 1, padding: 20, justifyContent: 'center',
alignItems: 'center' } },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 13, marginBottom:
12, textTransform: 'uppercase', letterSpacing: 1 } }, state.flipped ? '💡 Answer' : '?
Question'),
    React.createElement(TouchableOpacity, {
      onPress: handleFlip,
      style: {
        backgroundColor: state.flipped ? (cat ? cat.color + '22' : successColor + '22') :
cardColor,
        borderRadius: 24, padding: 32, width: '100%', minHeight: 220,
        alignItems: 'center', justifyContent: 'center',
        borderWidth: 2, borderColor: state.flipped ? (cat ? cat.color : successColor) :
cardBorder,
        shadowColor: state.flipped ? (cat ? cat.color : successColor) : primaryColor,
        shadowOffset: { width: 0, height: 8 }, shadowOpacity: 0.3, shadowRadius: 16,
elevation: 8
      },
      componentId: 'flash-card-flip'
    },
    React.createElement(Text, { style: { color: textPrimary, fontSize: 20, fontWeight: '700',
textAlign: 'center', lineHeight: 30 } },
      state.flipped ? currentCard.answer : currentCard.question
    ),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12, marginTop: 16
} }, 'Tap to ' + (state.flipped ? 'see question' : 'reveal answer'))
  ),
  state.flipped
    ? React.createElement(View, { style: { flexDirection: 'row', gap: 16, marginTop: 28,
width: '100%' } },
      React.createElement(TouchableOpacity, {
        onPress: function() { handleNext(false); },
        style: { flex: 1, backgroundColor: errorColor + '22', borderRadius: 16,
paddingVertical: 18, alignItems: 'center', borderWidth: 1, borderColor: errorColor + '55' },
        componentId: 'study-dont-know'
      },
      React.createElement(Text, { style: { fontSize: 22 } }, '😊'),

```

```

        React.createElement(Text, { style: { color: errorColor, fontWeight: '700', marginTop:
4 } }, 'Still learning')
    ),
    React.createElement(TouchableOpacity, {
        onPress: function() { handleNext(true); },
        style: { flex: 1, backgroundColor: successColor + '22', borderRadius: 16,
paddingVertical: 18, alignItems: 'center', borderWidth: 1, borderColor: successColor + '55' },
        componentId: 'study-know'
    },
        React.createElement(Text, { style: { fontSize: 22 } }, '🎯'),
        React.createElement(Text, { style: { color: successColor, fontWeight: '700',
marginTop: 4 } }, 'Got it!')
    )
)
: React.createElement(View, { style: { marginTop: 20, alignItems: 'center' } },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 14 } }, '👉 Tap
the card to reveal the answer')
)
)
);
};
// @end:StudySetScreen

```

```

// @section:GamesScreen-state @depends:[ThemeContext]
var useGamesState = function() {
    var themeCtx = useTheme();
    var theme = themeCtx.theme;
    var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });
    var prefs = prefsResult[0];
    var gameActiveState = useState(false);
    var gameActive = gameActiveState[0];
    var setGameActive = gameActiveState[1];
    var gameCardsState = useState([]);
    var gameCards = gameCardsState[0];
    var setGameCards = gameCardsState[1];
    var flippedState = useState([]);
    var flippedIds = flippedState[0];
    var setFlippedIds = flippedState[1];
    var matchedState = useState([]);
    var matchedIds = matchedState[0];
    var setMatchedIds = matchedState[1];
    var movesState = useState(0);
    var moves = movesState[0];
    var setMoves = movesState[1];
    var timerState = useState(0);
    var timer = timerState[0];
    var setTimer = timerState[1];
    var gameOverState = useState(false);

```

```

var gameOver = gameOverState[0];
var setGameOver = gameOverState[1];
var lockBoardState = useState(false);
var lockBoard = lockBoardState[0];
var setLockBoard = lockBoardState[1];
var selectedCatState = useState("");
var selectedCat = selectedCatState[0];
var setSelectedCat = selectedCatState[1];
var difficultyState = useState(4);
var difficulty = difficultyState[0];
var setDifficulty = difficultyState[1];

return { theme: theme, prefs: prefs, gameActive: gameActive, setGameActive:
setGameActive, gameCards: gameCards, setGameCards: setGameCards, flippedIds:
flippedIds, setFlippedIds: setFlippedIds, matchedIds: matchedIds, setMatchedIds:
setMatchedIds, moves: moves, setMoves: setMoves, timer: timer, setTimer: setTimer,
gameOver: gameOver, setGameOver: setGameOver, lockBoard: lockBoard, setLockBoard:
setLockBoard, selectedCat: selectedCat, setSelectedCat: setSelectedCat, difficulty: difficulty,
setDifficulty: setDifficulty };
};
// @end:GamesScreen-state

// @section:GamesScreen @depends:[GamesScreen-state,static-data]
var GamesScreen = function(props) {
  var state = useGamesState();
  var insets = useSafeAreaInsets();
  var scrollBottomPadding = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + SCROLL_EXTRA_PADDING);
  var sessionMutation = useMutation('study_sessions', 'insert');
  var sessionMutate = sessionMutation.mutate;
  var timerRef = useRef(null);

  useEffect(function() {
    if (state.gameActive && !state.gameOver) {
      timerRef.current = setInterval(function() { state.setTimer(function(prev) { return prev + 1;
}); }, 1000);
    } else {
      if (timerRef.current) { clearInterval(timerRef.current); }
    }
    return function() { if (timerRef.current) { clearInterval(timerRef.current); } };
  }, [state.gameActive, state.gameOver]);

  var startGame = function() {
    var catId = state.selectedCat || (state.prefs.categories[0] || 'anime');
    var emojiPool = THEME_EMOJI_PAIRS[catId] || THEME_EMOJI_PAIRS.anime;
    var pairs = emojiPool.slice(0, state.difficulty);
    var allCards = [];
    pairs.forEach(function(emoji, idx) {

```

```

    allCards.push({ id: 'a' + idx, pairId: idx, emoji: emoji });
    allCards.push({ id: 'b' + idx, pairId: idx, emoji: emoji });
  });
  for (var i = allCards.length - 1; i > 0; i--) {
    var j = Math.floor(Math.random() * (i + 1));
    var tmp = allCards[i];
    allCards[i] = allCards[j];
    allCards[j] = tmp;
  }
  state.setGameCards(allCards);
  state.setFlippedIds([]);
  state.setMatchedIds([]);
  state.setMoves(0);
  state.setTimer(0);
  state.setGameOver(false);
  state.setLockBoard(false);
  state.setGameActive(true);
};

var handleCardPress = function(card) {
  if (state.lockBoard) return;
  if (state.flippedIds.indexOf(card.id) >= 0) return;
  if (state.matchedIds.indexOf(card.pairId) >= 0) return;
  var newFlipped = state.flippedIds.concat([card.id]);
  state.setFlippedIds(newFlipped);
  if (newFlipped.length === 2) {
    state.setMoves(function(prev) { return prev + 1; });
    state.setLockBoard(true);
    var firstId = newFlipped[0];
    var secondId = newFlipped[1];
    var firstCard = state.gameCards.find(function(c) { return c.id === firstId; });
    var secondCard = state.gameCards.find(function(c) { return c.id === secondId; });
    if (firstCard && secondCard && firstCard.pairId === secondCard.pairId) {
      var newMatched = state.matchedIds.concat([firstCard.pairId]);
      state.setMatchedIds(newMatched);
      state.setFlippedIds([]);
      state.setLockBoard(false);
      if (newMatched.length === state.difficulty) {
        state.setGameOver(true);
        var score = Math.max(10, 100 - state.moves * 2);
        sessionMutate({ session_type: 'memory_game', content_id: state.selectedCat ||
'anime', duration_seconds: state.timer, score: score, session_date: new
Date().toISOString().slice(0, 10) });
      }
    } else {
      setTimeout(function() { state.setFlippedIds([]); state.setLockBoard(false); }, 1000);
    }
  }
};

```

```

};

var formatTime = function(s) {
  var m = Math.floor(s / 60);
  var sec = s % 60;
  return (m < 10 ? '0' : '') + m + ':' + (sec < 10 ? '0' : '') + sec;
};

var catForGame = PREFERENCE_CATEGORIES.find(function(c) { return c.id ===
(state.selectedCat || (state.prefs.categories[0] || 'anime')); });
var userCats = state.prefs && state.prefs.categories && state.prefs.categories.length > 0 ?
state.prefs.categories : PREFERENCE_CATEGORIES.map(function(c) { return c.id;
}).slice(0, 4);

if (state.gameOver) {
  var score = Math.max(10, 100 - state.moves * 2);
  return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor,
alignItems: 'center', justifyContent: 'center', padding: 24 }, componentId: 'game-over-screen'
},
    React.createElement(Text, { style: { fontSize: 72 } }, '🏆'),
    React.createElement(Text, { style: { color: textPrimary, fontSize: 28, fontWeight: '900',
marginTop: 12 } }, 'Puzzle Complete!'),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 15, marginTop: 8,
marginBottom: 24 } }, 'You matched all ' + state.difficulty + ' pairs!'),
    React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 24, width: '100%', marginBottom: 28, borderWidth: 1, borderColor: cardBorder,
flexDirection: 'row', justifyContent: 'space-around' } },
      React.createElement(View, { style: { alignItems: 'center' } },
        React.createElement(Text, { style: { fontSize: 32 } }, '🎯'),
        React.createElement(Text, { style: { color: accentColor, fontSize: 26, fontWeight: '900'
} }, String(score)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Score')
      ),
      React.createElement(View, { style: { alignItems: 'center' } },
        React.createElement(Text, { style: { fontSize: 32 } }, '👉'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 26, fontWeight: '900'
} }, String(state.moves)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Moves')
      ),
      React.createElement(View, { style: { alignItems: 'center' } },
        React.createElement(Text, { style: { fontSize: 32 } }, '🕒'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 26, fontWeight: '900'
} }, formatTime(state.timer)),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Time')
      )
    ),
    React.createElement(TouchableOpacity, { onPress: startGame, style: {
backgroundColor: primaryColor, borderRadius: 14, paddingVertical: 16, paddingHorizontal:

```

```

40, marginBottom: 12, width: '100%', alignItems: 'center' }, componentId:
'game-play-again-btn' },
    React.createElement(Text, { style: { color: '#fff', fontSize: 17, fontWeight: '800' } }, '🎮
Play Again')
    ),
    React.createElement(TouchableOpacity, { onPress: function() {
state.setGameActive(false); }, style: { backgroundColor: cardColor, borderRadius: 14,
paddingVertical: 16, paddingHorizontal: 40, borderWidth: 1, borderColor: cardBorder, width:
'100%', alignItems: 'center' }, componentId: 'game-menu-btn' },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 16, fontWeight:
'700' } }, '← Game Menu')
    )
  );
}

if (state.gameActive) {
  var cols = state.difficulty <= 4 ? 4 : 4;
  var cardW = (Dimensions.get('window').width - 40 - (cols - 1) * 10) / cols;
  return React.createElement(View, { style: { flex: 1, backgroundColor: backgroundColor },
componentId: 'game-active-screen' },
    React.createElement(View, { style: { paddingTop: insets.top + 10, paddingBottom: 10,
paddingHorizontal: 16, backgroundColor: cardColor, flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between', borderBottomWidth: 1, borderBottomColor: cardBorder } },
    React.createElement(TouchableOpacity, { onPress: function() {
state.setGameActive(false); }, style: { padding: 4 }, componentId: 'game-exit-btn' },
    React.createElement(Text, { style: { color: primaryColor, fontSize: 22 } }, '←')
    ),
    React.createElement(View, { style: { alignItems: 'center' } },
    React.createElement(Text, { style: { color: textPrimary, fontWeight: '800', fontSize: 16
} }, (catForGame ? catForGame.emoji : '🐱') + ' Memory Match'),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } },
state.matchedIds.length + '/' + state.difficulty + ' matched')
    ),
    React.createElement(View, { style: { alignItems: 'flex-end' } },
    React.createElement(Text, { style: { color: accentColor, fontWeight: '700' } },
formatTime(state.timer)),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 11 } },
state.moves + ' moves')
    )
    ),
    React.createElement(View, { style: { height: 4 } },
    React.createElement(View, { style: { height: 4, backgroundColor: primaryColor, width:
(state.matchedIds.length / state.difficulty * 100) + '%' } })
    ),
    React.createElement(ScrollView, { contentContainerStyle: { padding: 16,
paddingBottom: 40 }, showsVerticalScrollIndicator: false },
    React.createElement(View, { style: { flexDirection: 'row', flexWrap: 'wrap', gap: 10,
justifyContent: 'center' } },

```

```

state.gameCards.map(function(card) {
  var isFlipped = state.flippedIds.indexOf(card.id) >= 0;
  var isMatched = state.matchedIds.indexOf(card.pairId) >= 0;
  var isVisible = isFlipped || isMatched;
  return React.createElement(TouchableOpacity, {
    key: card.id,
    onPress: function() { handleCardPress(card); },
    style: {
      width: cardW, height: cardW,
      borderRadius: 14, alignItems: 'center', justifyContent: 'center',
      backgroundColor: isMatched ? successColor + '33' : isFlipped ? primaryColor +
'44' : cardColor,
      borderWidth: 2, borderColor: isMatched ? successColor : isFlipped ?
primaryColor : cardBorder
    },
    componentId: 'game-card-' + card.id
  },
  isVisible
    ? React.createElement(Text, { style: { fontSize: cardW * 0.4 } }, card.emoji)
    : React.createElement(Text, { style: { fontSize: cardW * 0.35, color:
textSecondary } }, '?')
  );
})
)
);
}

return React.createElement(ScrollView, {
  style: { flex: 1, backgroundColor: backgroundColor },
  contentContainerStyle: { paddingTop: insets.top + 16, paddingBottom:
scrollBottomPadding, paddingHorizontal: 16 },
  showsVerticalScrollIndicator: false,
  componentId: 'games-menu-screen'
},
  React.createElement(Text, { style: { color: textPrimary, fontSize: 24, fontWeight: '900',
marginBottom: 4 } }, '🎮 Games'),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom: 24
} }, 'Learn through play — powered by your interests!'),

  React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 20, marginBottom: 20, borderWidth: 1, borderColor: cardBorder }, componentId:
'memory-game-card' },
    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
marginBottom: 16 } },
      React.createElement(View, { style: { width: 52, height: 52, borderRadius: 16,
backgroundColor: primaryColor + '33', alignItems: 'center', justifyContent: 'center',
marginRight: 12 } },

```

```

    React.createElement(Text, { style: { fontSize: 28 } }, '🧠')
  ),
  React.createElement(View, { style: { flex: 1 } },
    React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800'
} }, 'Memory Match'),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 13 } }, 'Flip and
match themed pairs')
  )
),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom:
14 } }, '🎮 Choose your theme:'),
  React.createElement(ScrollView, { horizontal: true, showsHorizontalScrollIndicator:
false, style: { flexGrow: 'initial', marginBottom: 16 } },
    userCats.map(function(catId) {
      var cat2 = PREFERENCE_CATEGORIES.find(function(c) { return c.id === catId; });
      if (!cat2) return null;
      var isActive = state.selectedCat === catId || (!state.selectedCat &&
state.prefs.categories[0] === catId);
      return React.createElement(TouchableOpacity, {
        key: catId,
        onPress: function() { state.setSelectedCat(catId); },
        style: { backgroundColor: isActive ? cat2.color : cardBorder + '33', borderRadius:
12, paddingHorizontal: 14, paddingVertical: 10, marginRight: 10, alignItems: 'center',
borderWidth: 1, borderColor: isActive ? cat2.color : cardBorder },
        componentId: 'game-cat-' + catId
      },
        React.createElement(Text, { style: { fontSize: 22 } }, cat2.emoji),
        React.createElement(Text, { style: { color: isActive ? '#fff' : textSecondary, fontSize:
12, fontWeight: '600', marginTop: 4 } }, cat2.label)
      );
    })
  ),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom:
10 } }, '⚡ Difficulty:'),
  React.createElement(View, { style: { flexDirection: 'row', gap: 10, marginBottom: 18 } },
    [4, 6, 8].map(function(diff) {
      var labels = { 4: 'Easy', 6: 'Medium', 8: 'Hard' };
      var icons = { 4: '★', 6: '★★', 8: '★★★' };
      return React.createElement(TouchableOpacity, {
        key: diff,
        onPress: function() { state.setDifficulty(diff); },
        style: { flex: 1, backgroundColor: state.difficulty === diff ? primaryColor : cardBorder
+ '33', borderRadius: 12, paddingVertical: 10, alignItems: 'center', borderWidth: 1,
borderColor: state.difficulty === diff ? primaryColor : cardBorder },
        componentId: 'diff-btn-' + diff
      },
        React.createElement(Text, { style: { fontSize: 14 } }, icons[diff]),

```



```

        React.createElement(Text, { style: { color: state.difficulty === diff ? '#fff' :
textSecondary, fontWeight: '700', fontSize: 12 } }, labels[diff])
    );
    })
),
    React.createElement(TouchableOpacity, {
        onPress: startGame,
        style: { backgroundColor: primaryColor, borderRadius: 14, paddingVertical: 16,
alignItems: 'center' },
        componentId: 'start-game-btn'
    },
        React.createElement(Text, { style: { color: '#fff', fontSize: 16, fontWeight: '800' } }, '🚀
Start Game!')
    )
),

    React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 20, borderWidth: 1, borderColor: cardBorder, opacity: 0.7 }, componentId:
'coming-soon-card' },
        React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
marginBottom: 8 } },
            React.createElement(View, { style: { width: 52, height: 52, borderRadius: 16,
backgroundColor: accentColor + '33', alignItems: 'center', justifyContent: 'center',
marginRight: 12 } },
                React.createElement(Text, { style: { fontSize: 28 } }, '⚡')
            ),
            React.createElement(View, { style: { flex: 1 } },
                React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800'
} }, 'Speed Quiz'),
                React.createElement(Text, { style: { color: textSecondary, fontSize: 13 } }, 'Race
against the clock!')
            ),
            React.createElement(View, { style: { backgroundColor: accentColor + '33',
borderRadius: 8, paddingHorizontal: 10, paddingVertical: 4 } },
                React.createElement(Text, { style: { color: accentColor, fontSize: 12, fontWeight: '700'
} }, 'Soon')
            )
        ),
        React.createElement(Text, { style: { color: textSecondary, fontSize: 13 } }, 'Answer
themed trivia questions as fast as you can!')
    )
);
};
// @end:GamesScreen

// @section:ChallengesScreen-state @depends:[ThemeContext]
var useChallengesState = function() {
    var themeCtx = useTheme();

```

```

var theme = themeCtx.theme;
var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });
var prefs = prefsResult[0];
var today = new Date().toISOString().slice(0, 10);
var challengesQuery = useQuery('daily_challenges', {}, { column: 'challenge_date',
ascending: false });
var challenges = challengesQuery.data;
var challengesLoading = challengesQuery.loading;
var challengesRefetch = challengesQuery.refetch;
var insertMutation = useMutation('daily_challenges', 'insert');
var insertMutate = insertMutation.mutate;
var updateMutation = useMutation('daily_challenges', 'update');
var updateMutate = updateMutation.mutate;
var sessionMutation = useMutation('study_sessions', 'insert');
var sessionMutate = sessionMutation.mutate;
var selectedTabState = useState('today');
var selectedTab = selectedTabState[0];
var setSelectedTab = selectedTabState[1];
var todayChallengeState = useState(null);
var todayChallenge = todayChallengeState[0];
var setTodayChallenge = todayChallengeState[1];
var loadedRef = React.useRef(false);

useEffect(function() {
  if (challengesLoading || loadedRef.current) return;
  var todayEntry = null;
  if (challenges && challenges.length > 0) {
    todayEntry = challenges.find(function(c) { return c.challenge_date === today; });
  }
  if (!todayEntry) {
    var idx = (new Date().getDate()) % SOCIAL_CHALLENGES.length;
    var pick = SOCIAL_CHALLENGES[idx];
    var newChallenge = { challenge_text: pick.text, category: pick.category, completed:
false, completed_date: null, challenge_date: today };
    insertMutate(newChallenge).then(function() { challengesRefetch(); });
  }
  setTodayChallenge(todayEntry);
  loadedRef.current = true;
}, [challenges, challengesLoading]);

useEffect(function() {
  if (!challenges) return;
  var entry = challenges.find(function(c) { return c.challenge_date === today; });
  setTodayChallenge(entry || null);
}, [challenges]);

return { theme: theme, prefs: prefs, challenges: challenges, challengesLoading:
challengesLoading, challengesRefetch: challengesRefetch, updateMutate: updateMutate,

```

```

sessionMutate: sessionMutate, selectedTab: selectedTab, setSelectedTab: setSelectedTab,
todayChallenge: todayChallenge, today: today };
};
// @end:ChallengesScreen-state

// @section:ChallengesScreen @depends:[ChallengesScreen-state,static-data]
var ChallengesScreen = function() {
  var state = useChallengesState();
  var insets = useSafeAreaInsets();
  var paddingBottom = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + SCROLL_EXTRA_PADDING);

  var handleComplete = function(challenge) {
    if (!challenge || !challenge.id) return;
    state.updateMutate({ id: challenge.id, data: { completed: true, completed_date:
state.today } })
    .then(function() {
      state.sessionMutate({ session_type: 'challenge', content_id: challenge.category,
duration_seconds: 0, score: 15, session_date: state.today });
      state.challengesRefetch();
    });
  };

  var completedCount = useMemo(function() {
    if (!state.challenges) return 0;
    return state.challenges.filter(function(c) { return c.completed; }).length;
  }, [state.challenges]);

  var streakCount = useMemo(function() {
    if (!state.challenges || state.challenges.length === 0) return 0;
    var completed = state.challenges.filter(function(c) { return c.completed; });
    completed.sort(function(a, b) { return b.challenge_date < a.challenge_date ? -1 : 1; });
    var count = 0;
    var checkDate = new Date();
    for (var i = 0; i < completed.length; i++) {
      var d = checkDate.toISOString().slice(0, 10);
      if (completed[i].challenge_date === d) { count++;
checkDate.setDate(checkDate.getDate() - 1); }
      else { break; }
    }
    return count;
  }, [state.challenges]);

  var socialChallenge = SOCIAL_CHALLENGES.find(function(c) { return
state.todayChallenge && c.text === state.todayChallenge.challenge_text; });
  var challengeEmoji = socialChallenge ? socialChallenge.emoji : '🌟';

  return React.createElement(ScrollView, {

```

```

    style: { flex: 1, backgroundColor: backgroundColor },
    contentContainerStyle: { paddingTop: insets.top + 16, paddingBottom:
scrollBottomPadding, paddingHorizontal: 16 },
    showsVerticalScrollIndicator: false,
    componentId: 'challenges-screen'
  },
  React.createElement(Text, { style: { color: textPrimary, fontSize: 24, fontWeight: '900',
marginBottom: 4 } }, '🏆 Daily Challenges'),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom: 20
} }, 'Build social confidence one challenge at a time!'),

  React.createElement(View, { style: { flexDirection: 'row', gap: 12, marginBottom: 24 } },
    React.createElement(View, { style: { flex: 1, backgroundColor: primaryColor + '22',
borderRadius: 16, padding: 16, alignItems: 'center', borderWidth: 1, borderColor:
primaryColor + '44' } },
      React.createElement(Text, { style: { fontSize: 28 } }, '🔥'),
      React.createElement(Text, { style: { color: accentColor, fontSize: 24, fontWeight: '900' }
}, String(streakCount)),
      React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Day Streak')
    ),
    React.createElement(View, { style: { flex: 1, backgroundColor: successColor + '22',
borderRadius: 16, padding: 16, alignItems: 'center', borderWidth: 1, borderColor:
successColor + '44' } },
      React.createElement(Text, { style: { fontSize: 28 } }, '✅'),
      React.createElement(Text, { style: { color: successColor, fontSize: 24, fontWeight:
'900' } }, String(completedCount)),
      React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'Completed')
    ),
    React.createElement(View, { style: { flex: 1, backgroundColor: accentColor + '22',
borderRadius: 16, padding: 16, alignItems: 'center', borderWidth: 1, borderColor:
accentColor + '44' } },
      React.createElement(Text, { style: { fontSize: 28 } }, '★'),
      React.createElement(Text, { style: { color: accentColor, fontSize: 24, fontWeight: '900' }
}, String(completedCount * 15)),
      React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } }, 'XP Earned')
    )
  ),

  React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '📅 Today's Challenge'),
  state.challengesLoading
    ? React.createElement(ActivityIndicator, { color: primaryColor, size: 'large', style: {
marginVertical: 40 } })
    : state.todayChallenge
    ? React.createElement(View, {
      style: {
        backgroundColor: state.todayChallenge.completed ? successColor + '22' :
primaryColor + '22',

```

```

        borderRadius: 20, padding: 24, marginBottom: 24,
        borderWidth: 2, borderColor: state.todayChallenge.completed ? successColor :
primaryColor
    },
    componentId: 'today-challenge-card'
  },
  React.createElement(View, { style: { flexDirection: 'row', alignItems: 'flex-start',
marginBottom: 16 } }),
  React.createElement(View, { style: { width: 56, height: 56, borderRadius: 28,
backgroundColor: state.todayChallenge.completed ? successColor + '33' : primaryColor +
'33', alignItems: 'center', justifyContent: 'center', marginRight: 14 } }),
  React.createElement(Text, { style: { fontSize: 30 } }, challengeEmoji)
),
  React.createElement(View, { style: { flex: 1 } },
    React.createElement(Text, { style: { color: state.todayChallenge.completed ?
successColor : primaryColor, fontSize: 12, fontWeight: '700', textTransform: 'uppercase',
letterSpacing: 1, marginBottom: 4 } }, state.todayChallenge.category),
    React.createElement(Text, { style: { color: textPrimary, fontSize: 16, fontWeight:
'700', lineHeight: 24 } }, state.todayChallenge.challenge_text)
  ),
  state.todayChallenge.completed
    ? React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'center', gap: 8 } },
      React.createElement(Text, { style: { fontSize: 24 } }, '🎉'),
      React.createElement(Text, { style: { color: successColor, fontSize: 16,
fontWeight: '800' } }, 'Challenge Complete! +15 XP')
    )
    : React.createElement(TouchableOpacity, {
      onPress: function() { handleComplete(state.todayChallenge); },
      style: { backgroundColor: primaryColor, borderRadius: 14, paddingVertical: 14,
alignItems: 'center' },
      componentId: 'complete-challenge-btn'
    },
      React.createElement(Text, { style: { color: '#fff', fontSize: 15, fontWeight: '800' }
}, '✅ Mark as Complete! (+15 XP)')
    )
  ),
  React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 16,
padding: 24, alignItems: 'center', marginBottom: 24, borderWidth: 1, borderColor: cardBorder
} },
    React.createElement(Text, { style: { fontSize: 40, marginBottom: 8 } }, '⌚'),
    React.createElement(Text, { style: { color: textSecondary, textAlign: 'center' } },
>Loading today's challenge...')
  ),

  React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '📖 What makes a great social challenge?'),

```

```

    React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 16,
padding: 18, marginBottom: 16, borderWidth: 1, borderColor: cardBorder } },
    [🗣️ Speak up in class at least once', '👂 Listen without interrupting', '🤝 Help a friend
with homework', '💬 Start a conversation about shared interests', '🙏 Thank someone for
their help today'].map(function(tip, i) {
    return React.createElement(View, { key: String(i), style: { flexDirection: 'row',
marginBottom: 10 } },
    React.createElement(Text, { style: { color: textSecondary, fontSize: 14, lineHeight:
22, flex: 1 } }, tip)
    );
    })
    ),

```

```

    state.challenges && state.challenges.filter(function(c) { return c.challenge_date !==
state.today; }).length > 0
    ? React.createElement(View, { componentId: 'past-challenges' },
    React.createElement(Text, { style: { color: textPrimary, fontSize: 18, fontWeight: '800',
marginBottom: 12 } }, '📖 Past Challenges'),
    state.challenges.filter(function(c) { return c.challenge_date !== state.today; }).slice(0,
5).map(function(ch) {
    return React.createElement(View, {
    key: ch.id || ch.challenge_date,
    style: { backgroundColor: cardColor, borderRadius: 14, padding: 14,
marginBottom: 10, flexDirection: 'row', alignItems: 'center', borderWidth: 1, borderColor:
ch.completed ? successColor + '44' : cardBorder },
    componentId: 'past-challenge-' + ch.id
    },
    React.createElement(Text, { style: { fontSize: 22, marginRight: 12 } }, ch.completed
? '✅' : '❌'),
    React.createElement(View, { style: { flex: 1 } },
    React.createElement(Text, { style: { color: textPrimary, fontSize: 14, fontWeight:
'600' } }, ch.challenge_text),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 12 } },
ch.challenge_date)
    )
    );
    })
    )
    : null
    );
};
// @end:ChallengesScreen

```

```

// @section:SettingsScreen-state @depends:[ThemeContext]
var useSettingsState = function() {
    var themeCtx = useTheme();
    var theme = themeCtx.theme;
    var prefsResult = useStorage('user_preferences', { categories: [], selectedThemes: [] });

```

```

var prefs = prefsResult[0];
var setPrefs = prefsResult[1];
var localPrefsState = useState(null);
var localPrefs = localPrefsState[0];
var setLocalPrefs = localPrefsState[1];
var editingState = useState(false);
var editing = editingState[0];
var setEditing = editingState[1];

useEffect(function() {
  if (prefs) { setLocalPrefs(prefs.categories || []); }
}, [prefs]);

return { theme: theme, prefs: prefs, setPrefs: setPrefs, localPrefs: localPrefs,
setLocalPrefs: setLocalPrefs, editing: editing, setEditing: setEditing };
};
// @end:SettingsScreen-state

// @section:SettingsScreen @depends:[SettingsScreen-state,static-data]
var SettingsScreen = function(props) {
  var navigation = props.navigation;
  var state = useSettingsState();
  var insets = useSafeAreaInsets();
  var scrollTopPadding = Platform.OS === 'web' ? WEB_TAB_MENU_PADDING :
(TAB_MENU_HEIGHT + insets.bottom + SCROLL_EXTRA_PADDING);
  var completeStorage = useStorage('onboarding_complete', false);
  var setComplete = completeStorage[1];

  var toggleLocal = function(id) {
    state.setLocalPrefs(function(prev) {
      if (!prev) return [id];
      if (prev.indexOf(id) >= 0) { return prev.filter(function(x) { return x !== id; }); }
      return prev.concat([id]);
    });
  };

  var handleSavePrefs = function() {
    var cats = state.localPrefs || [];
    state.setPrefs({ categories: cats, selectedThemes: cats });
    state.setEditing(false);
  };

  var handleReset = function() {
    if (Platform.OS === 'web') {
      if (!window.confirm('Reset all progress? This cannot be undone.')) return;
      setComplete(false).then(function() { navigation.replace('Onboarding'); });
    } else {

```

```

Alert.alert('Reset Progress', 'This will clear all your data and restart the app. Are you
sure?', [
  { text: 'Cancel', style: 'cancel' },
  { text: 'Reset', style: 'destructive', onPress: function() {
setComplete(false).then(function() { navigation.replace('Onboarding'); }); } }
]);
}
};

var selectedCats = state.prefs ? (state.prefs.categories || []) : [];

return React.createElement(ScrollView, {
  style: { flex: 1, backgroundColor: backgroundColor },
  contentContainerStyle: { paddingTop: insets.top + 16, paddingBottom:
scrollBottomPadding, paddingHorizontal: 16 },
  showsVerticalScrollIndicator: false,
  componentId: 'settings-screen'
},
  React.createElement(Text, { style: { color: textPrimary, fontSize: 24, fontWeight: '900',
marginBottom: 4 } }, '⚙️ Settings'),
  React.createElement(Text, { style: { color: textSecondary, fontSize: 14, marginBottom: 24
} }, 'Customize your LearnFlow experience'),

  React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 20, marginBottom: 20, borderWidth: 1, borderColor: cardBorder }, componentId:
'prefs-section' },
    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center',
justifyContent: 'space-between', marginBottom: 16 } },
      React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center', gap: 8 } },
        React.createElement(Text, { style: { fontSize: 22 } }, '🎨'),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 17, fontWeight: '800'
} }, 'My Interests')
      ),
      React.createElement(TouchableOpacity, {
        onPress: function() { state.setEditing(function(prev) { return !prev; }); },
        style: { backgroundColor: state.editing ? errorColor + '33' : primaryColor + '33',
borderRadius: 10, paddingHorizontal: 14, paddingVertical: 8 },
        componentId: 'edit-prefs-btn'
      },
        React.createElement(Text, { style: { color: state.editing ? errorColor : primaryColor,
fontWeight: '700' } }, state.editing ? 'Cancel' : 'Edit')
      )
    ),
    state.editing
    ? React.createElement(View, {},
      React.createElement(View, { style: { flexDirection: 'row', flexWrap: 'wrap', gap: 10,
marginBottom: 16 } },
        PREFERENCE_CATEGORIES.map(function(cat) {

```



```

    var isSelected = state.localPrefs && state.localPrefs.indexOf(cat.id) >= 0;
    return React.createElement(TouchableOpacity, {
      key: cat.id,
      onPress: function() { toggleLocal(cat.id); },
      style: { backgroundColor: isSelected ? cat.color + '33' : backgroundColor,
borderRadius: 12, paddingHorizontal: 14, paddingVertical: 10, borderWidth: 1, borderColor:
isSelected ? cat.color : cardBorder, flexDirection: 'row', alignItems: 'center', gap: 6 },
      componentId: 'settings-cat-' + cat.id
    },
    React.createElement(Text, { style: { fontSize: 18 } }, cat.emoji),
    React.createElement(Text, { style: { color: isSelected ? cat.color :
textSecondary, fontWeight: '600', fontSize: 13 } }, cat.label)
  );
})
),
React.createElement(TouchableOpacity, {
  onPress: handleSavePrefs,
  style: { backgroundColor: primaryColor, borderRadius: 12, paddingVertical: 12,
alignItems: 'center' },
  componentId: 'save-prefs-btn'
},
  React.createElement(Text, { style: { color: '#fff', fontWeight: '700', fontSize: 15 } },
' Save Preferences')
)
)
: React.createElement(View, { style: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 } },
  selectedCats.map(function(catId) {
    var cat = PREFERENCE_CATEGORIES.find(function(c) { return c.id === catId; });
    if (!cat) return null;
    return React.createElement(View, {
      key: catId,
      style: { backgroundColor: cat.color + '22', borderRadius: 10, paddingHorizontal:
12, paddingVertical: 6, borderWidth: 1, borderColor: cat.color + '55', flexDirection: 'row',
alignItems: 'center', gap: 4 },
      componentId: 'settings-selected-' + catId
    },
    React.createElement(Text, { style: { fontSize: 14 } }, cat.emoji),
    React.createElement(Text, { style: { color: cat.color, fontWeight: '600', fontSize:
13 } }, cat.label)
  );
})
)
),

  React.createElement(View, { style: { backgroundColor: cardColor, borderRadius: 20,
padding: 20, marginBottom: 20, borderWidth: 1, borderColor: cardBorder }, componentId:
'about-section' },

```

```

    React.createElement(View, { style: { flexDirection: 'row', alignItems: 'center', gap: 10,
marginBottom: 14 } },
    React.createElement(Text, { style: { fontSize: 22 } }, '🌊'),
    React.createElement(Text, { style: { color: textPrimary, fontSize: 17, fontWeight: '800' }
}, 'About LearnFlow')
  ),
  React.createElement(View, { style: { gap: 8 } },
    [['Version', '1.0.0'], ['Made for', 'Students who love to learn! 🎓'], ['Theme', 'Dark mode
with vibrant accents']].map(function(row) {
      return React.createElement(View, { key: row[0], style: { flexDirection: 'row',
justifyContent: 'space-between', paddingVertical: 6, borderBottomWidth: 1,
borderBottomColor: cardBorder } },
        React.createElement(Text, { style: { color: textSecondary, fontSize: 14 } }, row[0]),
        React.createElement(Text, { style: { color: textPrimary, fontSize: 14, fontWeight:
'600' } }, row[1])
      );
    })
  )
),

```

```

    React.createElement(View, { style: { backgroundColor: errorColor + '11', borderRadius:
16, padding: 20, borderWidth: 1, borderColor: errorColor + '33' }, componentId: 'danger-zone'
},
    React.createElement(Text, { style: { color: errorColor, fontSize: 16, fontWeight: '800',
marginBottom: 8 } }, '⚠️ Danger Zone'),
    React.createElement(Text, { style: { color: textSecondary, fontSize: 13, marginBottom:
14 } }, 'Resetting progress will clear all flashcard sets, study sessions, and challenges
permanently.'),
    React.createElement(TouchableOpacity, {
      onPress: handleReset,
      style: { backgroundColor: errorColor + '22', borderRadius: 12, paddingVertical: 14,
alignItems: 'center', borderWidth: 1, borderColor: errorColor },
      componentId: 'reset-btn'
    },
      React.createElement(Text, { style: { color: errorColor, fontSize: 15, fontWeight: '700' }
}, '🔄 Reset All Progress')
    )
  );
};
// @end:SettingsScreen

```

```

// @section:TabNavigator
@depends:[HomeScreen,FlashcardsScreen,GamesScreen,ChallengesScreen,SettingsScre
en,navigation-setup]
var TabNavigator = function() {
  var insets = useSafeAreaInsets();
  var themeCtx = useTheme();

```

```

var theme = themeCtx.theme;
return React.createElement(View, { style: { flex: 1, width: '100%', height: '100%', overflow:
'hidden' } },
  React.createElement(Tab.Navigator, {
    screenOptions: {
      headerShown: false,
      tabBarStyle: {
        position: 'absolute', bottom: 0,
        height: Platform.OS === 'web' ? TAB_MENU_HEIGHT : TAB_MENU_HEIGHT +
insets.bottom,
        paddingBottom: 0, borderTopWidth: 0,
        backgroundColor: cardColor, borderTopColor: cardBorder
      },
      tabBarItemStyle: { padding: 0 },
      tabBarActiveTintColor: primaryColor,
      tabBarInactiveTintColor: textSecondary,
      tabBarLabelStyle: { fontSize: 11, fontWeight: '600' }
    }
  },
    React.createElement(Tab.Screen, {
      name: 'Home',
      component: HomeScreen,
      options: {
        tabBarIcon: function(p) { return React.createElement(Ionicons, { name: p.focused ?
'home' : 'home-outline', size: 22, color: p.color }); },
        tabBarLabel: 'Home'
      }
    }),
    React.createElement(Tab.Screen, {
      name: 'Study',
      component: FlashcardsScreen,
      options: {
        tabBarIcon: function(p) { return React.createElement(Ionicons, { name: p.focused ?
'book' : 'book-outline', size: 22, color: p.color }); },
        tabBarLabel: 'Study'
      }
    }),
    React.createElement(Tab.Screen, {
      name: 'Games',
      component: GamesScreen,
      options: {
        tabBarIcon: function(p) { return React.createElement(Ionicons, { name: p.focused ?
'game-controller' : 'game-controller-outline', size: 22, color: p.color }); },
        tabBarLabel: 'Games'
      }
    }),
    React.createElement(Tab.Screen, {
      name: 'Challenges',

```

```

        component: ChallengesScreen,
        options: {
          tabBarIcon: function(p) { return React.createElement(Ionicons, { name: p.focused ?
'trophy' : 'trophy-outline', size: 22, color: p.color }); },
          tabBarLabel: 'Quests'
        }
      })),
      React.createElement(Tab.Screen, {
        name: 'Settings',
        component: SettingsScreen,
        options: {
          tabBarIcon: function(p) { return React.createElement(Ionicons, { name: p.focused ?
'settings' : 'settings-outline', size: 22, color: p.color }); },
          tabBarLabel: 'Settings'
        }
      })
    )
  );
};

var MainApp = function() {
  return React.createElement(TabNavigator);
};
// @end:TabNavigator

// @section:styles @depends:[theme]
var styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: backgroundColor },
  headerTitle: { color: textPrimary, fontSize: 18, fontWeight: '800' },
  card: { backgroundColor: cardColor, borderRadius: 16, padding: 16, borderWidth: 1,
borderColor: cardBorder },
  primaryBtn: { backgroundColor: primaryColor, borderRadius: 14, paddingVertical: 14,
alignItems: 'center' },
  primaryBtnText: { color: '#fff', fontSize: 16, fontWeight: '700' },
  sectionTitle: { color: textPrimary, fontSize: 18, fontWeight: '800', marginBottom: 12 },
  bodyText: { color: textSecondary, fontSize: 14, lineHeight: 22 },
  fab: { position: 'absolute', right: 20, width: 58, height: 58, borderRadius: 29,
backgroundColor: primaryColor, alignItems: 'center', justifyContent: 'center', elevation: 8,
shadowColor: primaryColor, shadowOffset: { width: 0, height: 4 }, shadowOpacity: 0.5,
shadowRadius: 8 }
});
// @end:styles

// @section:return
@depends:[ThemeProvider,TabNavigator,OnboardingScreen,RootScreen,CreateSetScreen,
StudySetScreen]
return React.createElement(ThemeProvider, null,
  React.createElement(View, { style: { flex: 1, width: '100%', height: '100%' } },

```

```

    React.createElement(StatusBar, { barStyle: 'light-content', backgroundColor:
backgroundColor })),
    React.createElement(Stack.Navigator, {
      initialRouteName: 'Root',
      screenOptions: { headerShown: false, cardStyle: { backgroundColor: backgroundColor }
    }
  },
    React.createElement(Stack.Screen, { name: 'Root', component: RootScreen })),
    React.createElement(Stack.Screen, { name: 'Onboarding', component:
OnboardingScreen })),
    React.createElement(Stack.Screen, { name: 'Main', component: MainApp })),
    React.createElement(Stack.Screen, { name: 'CreateSet', component: CreateSetScreen
  })),
    React.createElement(Stack.Screen, { name: 'StudySet', component: StudySetScreen,
initialParams: { setId: null } })
  )
)
);
// @end:return
};
return ComponentFuncio

```